

Rapport Technique — Projet Billetix

Gestion des Événements — Architecture Orientée Services

Hadji Mehdi

Talamahcen Warda

Ziane Khaoula

Avril 2026 | Groupe 3 — Spécialité : Génie Logiciel | Master 1 — Architecture Orientée Services

Table des matières

1	Introduction	3
1.1	Contexte du Projet	3
1.2	Objectifs Techniques	3
2	Architecture du Système	4
2.1	Vue d'Ensemble	4
2.2	Diagramme de Séquence — Flux de Réservation	4
2.3	Diagramme d'Activité — Processus de Réservation	5
3	Description des Microservices	6
3.1	Auth Service	6
3.1.1	Endpoints	6
3.1.2	Modèle Utilisateur	6
3.1.3	Sécurité	6
3.2	Event Service	6
3.2.1	Endpoints	6
3.2.2	Modèle Événement	7
3.3	Booking Service	7
3.3.1	Endpoints	7
3.3.2	Modèle Réservation	7
3.3.3	Message RabbitMQ publié après réservation	7
4	Modélisation UML	9
4.1	Diagramme de Classes	9
4.2	Frontend	9
5	Guide de Tests Postman	10
5.1	Configuration de l'Environnement	10
5.2	Tests — Auth Service	10
5.2.1	Test 1 : Register	10
5.2.2	Test 2 : Login	10
5.2.3	Test 3 : Profil (Me)	11
5.2.4	Test 4 : Refresh Token	11

5.2.5	Test 5 : Sécurité — Accès refusé sans token	12
5.3	Tests — Event Service	12
5.3.1	Test 6 : Créer un Événement	12
5.3.2	Test 7 : Lister les Événements	13
5.3.3	Test 8 : Détail d'un Événement	13
5.3.4	Test 9 : Refus de Création (CLIENT)	13
5.4	Tests — Booking Service	13
5.4.1	Test 10 : Créer une Réservation	13
5.4.2	Test 11 : Mes Réservations	14
6	Infrastructure et Déploiement	15
6.1	Docker Compose	15
6.1.1	Démarrage complet	15
6.1.2	Vérification des conteneurs actifs	15
6.1.3	Services déployés	15
6.2	Tableau de Bord	15
6.3	Consul — Service Registry	15
6.4	Traefik — Reverse Proxy	15
6.5	RabbitMQ — Communication Asynchrone	16
7	Conclusion	17

1 Introduction

1.1 Contexte du Projet

Billetix est une plateforme de gestion et de réservation d'événements développée dans le cadre du cours d'Architecture Orientée Services (SOA). L'objectif est de concevoir une application distribuée reposant sur des microservices indépendants communicants de manière synchrone (REST) et asynchrone (RabbitMQ).

1.2 Objectifs Techniques

Contrainte	Technologie	Statut
API REST CRUD	Django REST Framework	Implémenté
Authentification JWT	Python / bcrypt	Implémenté
Service Registry / Discovery	HashiCorp Consul	Implémenté
Reverse Proxy / Load Balancing	Traefik v2	Implémenté
Communication Asynchrone	RabbitMQ	Implémenté
Déploiement Multi-Conteneurs	Docker Compose	Implémenté
Interface Utilisateur	React + TypeScript	Implémenté

2 Architecture du Système

2.1 Vue d'Ensemble

L'architecture de Billetix est composée de **8 services Docker** interconnectés au sein d'un réseau `billetix-network`.

- **Traefik** (port 80/8080) : point d'entrée unique, reverse proxy dynamique
- **Auth-Service** (port interne 8000) : authentification et gestion JWT
- **Event-Service** (port interne 8001) : CRUD des événements
- **Booking-Service** (port interne 8002) : gestion des réservations
- **Email-Worker** : consommateur RabbitMQ pour les notifications email
- **Frontend** : application React/TypeScript servie par Nginx
- **Consul** (port 8500) : service registry et health checks
- **RabbitMQ** (ports 5672/15672) : message broker asynchrone

2.2 Diagramme de Séquence — Flux de Réservation

Le diagramme ci-dessous illustre les interactions entre les différents microservices lors d'un scénario complet de réservation.

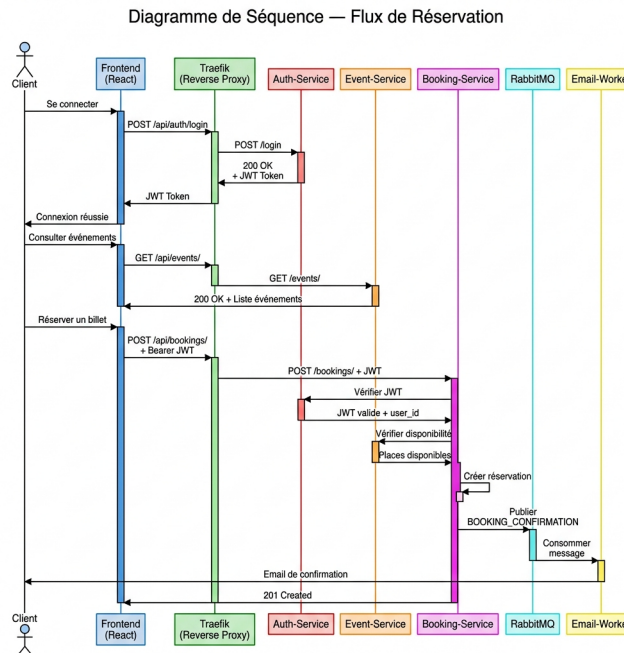


Figure 1: Diagramme de Séquence

Scénario :

1. L'utilisateur s'inscrit/se connecte : **Auth-Service** retourne un JWT
2. Il consulte les événements : **Event-Service** retourne la liste
3. Il réserve un billet : **Booking-Service** crée la réservation
4. Booking-Service publie dans RabbitMQ (queue `email.send`)
5. **Email-Worker** consomme le message et envoie la confirmation

2.3 Diagramme d'Activité — Processus de Réservation

Ce diagramme présente le flux d'activités côté client, système API et worker email, avec les décisions et les bifurcations possibles.

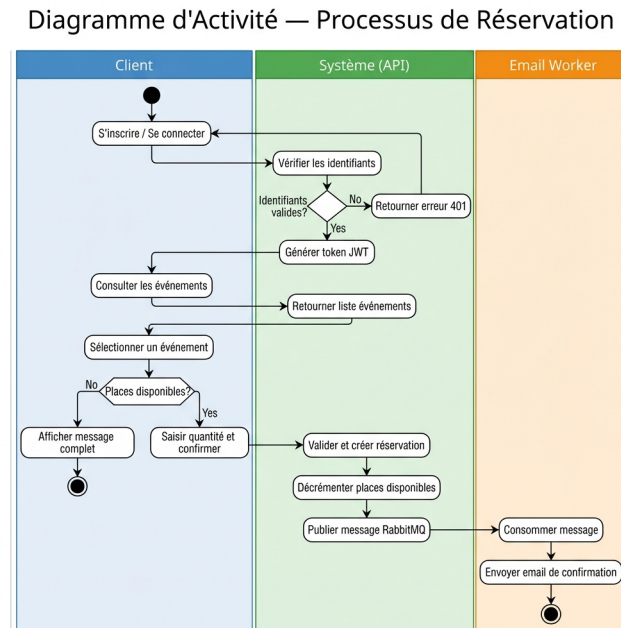


Figure 2: Diagramme d'Activité

3 Description des Microservices

3.1 Auth Service

Port interne : 8000 | Route Traefik : /api/auth

3.1.1 Endpoints

Méthode	Endpoint	Description	Auth
POST	/api/auth/register/	Inscription	Non
POST	/api/auth/login/	Connexion JWT	Non
POST	/api/auth/refresh/	Renouvellement token	Non
GET	/api/auth/me/	Profil utilisateur	Oui
GET	/api/auth/health/	Health check	Non

3.1.2 Modèle Utilisateur

Champ	Type	Description
id	UUID	Identifiant unique
email	String	Email (unique)
password	String	Hash bcrypt
first_name	String	Prénom
last_name	String	Nom
role	Enum	CLIENT ou ORGANIZER
is_active	Boolean	Compte actif
created_at	DateTime	Date de création

3.1.3 Sécurité

- **Hachage** : bcrypt avec salage automatique
- **Anti-brute force** : blocage après 5 tentatives échouées (300 secondes)
- **Tokens** : Access token (courte durée) + Refresh token (longue durée)

3.2 Event Service

Port interne : 8001 | Route Traefik : /api/events

3.2.1 Endpoints

Méthode	Endpoint	Description	Rôle requis
GET	/api/events/	Lister les événements	Public
POST	/api/events/	Créer un événement	ORGANIZER
GET	/api/events/{id}/	Détail d'un événement	Public
PUT	/api/events/{id}/	Modifier un événement	ORGANIZER

Méthode	Endpoint	Description	Rôle requis
DELETE	/api/events/{id}/	Supprimer	ORGANIZER

3.2.2 Modèle Événement

Champ	Type	Description
id	UUID	Identifiant unique
title	String	Titre (max 200 car.)
description	Text	Description
date	DateTime	Date de l'événement
location	String	Lieu
capacity	Integer	Capacité totale
available_seats	Integer	Places disponibles
price	Decimal	Prix par billet
category	String	Catégorie
status	Enum	DRAFT, PUBLISHED, CANCELLED
organizer_id	UUID	Référence organisateur

3.3 Booking Service

Port interne : 8002 | Route Traefik : /api/bookings

3.3.1 Endpoints

Méthode	Endpoint	Description	Auth
POST	/api/bookings/	Créer une réservation	Oui
GET	/api/bookings/me/	Mes réservations	Oui
GET	/api/bookings/health/	Health check	Non

3.3.2 Modèle Réservation

Champ	Type	Description
id	UUID	Identifiant unique
event_id	UUID	Référence à l'événement
user_id	UUID	Référence à l'utilisateur
quantity	Integer	Nombre de billets
total_price	Decimal	Prix total calculé
status	Enum	PENDING, CONFIRMED, CANCELLED
created_at	DateTime	Date de création

3.3.3 Message RabbitMQ publié après réservation

```
{  
  "type": "BOOKING_CONFIRMATION",  
  "to": "client@example.com",  
  "booking_id": "uuid...",  
  "event_title": "Festival Tech Alger 2025"  
}
```


4 Modélisation UML

4.1 Diagramme de Classes

Le diagramme de classes ci-dessous représente les entités principales du système Billetix, leurs attributs, leurs méthodes et leurs associations.

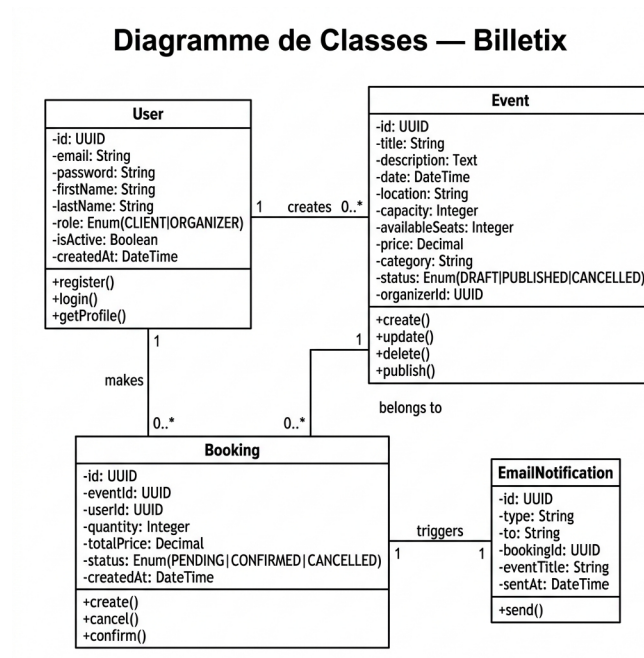


Figure 3: Diagramme de Classes

4.2 Frontend

Application React + TypeScript servie via Nginx.

Route	Description
/	Liste des événements publics
/login	Connexion utilisateur
/register	Inscription
/events	Gestion événements (ORGANIZER)
/bookings	Mes réservations (CLIENT)

5 Guide de Tests Postman

5.1 Configuration de l'Environnement

Dans Postman, allez dans **Environments** > **New** et créez ces variables :

Variable	Valeur initiale
BASE_URL	http://localhost
ACCESS_TOKEN	(vide)
REFRESH_TOKEN	(vide)
EVENT_ID	(vide)

5.2 Tests — Auth Service

5.2.1 Test 1 : Register

Paramètre	Valeur
Méthode	POST
URL	{{BASE_URL}}/api/auth/register/
Header	Content-Type: application/json

Body (raw > JSON) :

```
{
  "email": "organisateur@billetix.com",
  "password": "Secure123",
  "firstName": "Alice",
  "lastName": "Dupont",
  "role": "ORGANIZER"
}
```

Réponse attendue : 201 Created

Script Tests (onglet Tests) :

```
pm.test("Status 201", () => pm.response.to.have.status(201));
const json = pm.response.json();
pm.environment.set("ACCESS_TOKEN", json.access);
pm.environment.set("REFRESH_TOKEN", json.refresh);
```

5.2.2 Test 2 : Login

Paramètre	Valeur
Méthode	POST

Paramètre	Valeur
URL	{{BASE_URL}}/api/auth/login/
Header	Content-Type: application/json

Body : { "email": "organisateur@billetix.com", "password": "Secure123" }

Réponse attendue : 200 OK

Script Tests :

```
pm.test("Status 200", () => pm.response.to.have.status(200));
const json = pm.response.json();
pm.environment.set("ACCESS_TOKEN", json.access);
pm.environment.set("REFRESH_TOKEN", json.refresh);
```

5.2.3 Test 3 : Profil (Me)

Paramètre	Valeur
Méthode	GET
URL	{{BASE_URL}}/api/auth/me/
Header	Authorization: Bearer {{ACCESS_TOKEN}}

Réponse attendue : 200 OK

Script Tests :

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("Rôle correct", () => {
  pm.expect(pm.response.json().role).to.eql("ORGANIZER");
});
```

5.2.4 Test 4 : Refresh Token

Paramètre	Valeur
Méthode	POST
URL	{{BASE_URL}}/api/auth/refresh/

Body : { "refresh": "{{REFRESH_TOKEN}}" }

Script Tests :

```
pm.test("Nouveau token reçu", () => {
  const json = pm.response.json();
  pm.expect(json.access).to.be.a('string');
});
```

```
pm.environment.set("ACCESS_TOKEN", json.access);
});
```

5.2.5 Test 5 : Sécurité — Accès refusé sans token

Paramètre	Valeur
Méthode	GET
URL	{{BASE_URL}}/api/auth/me/
Header	(aucun Authorization)

Réponse attendue : 401 Unauthorized

5.3 Tests — Event Service

5.3.1 Test 6 : Créer un Événement

Paramètre	Valeur
Méthode	POST
URL	{{BASE_URL}}/api/events/
Headers	Content-Type: application/json + Authorization: Bearer {{ACCESS_TOKEN}}

Body :

```
{
  "title": "Festival Tech Alger 2025",
  "description": "La plus grande conférence tech d'Algérie",
  "date": "2025-09-15T10:00:00Z",
  "location": "Alger Convention Center",
  "capacity": 500,
  "price": 1500.00,
  "category": "Technologie"
}
```

Réponse attendue : 201 Created

Script Tests :

```
pm.test("Événement créé", () => pm.response.to.have.status(201));
pm.environment.set("EVENT_ID", pm.response.json().id);
```

5.3.2 Test 7 : Lister les Événements

Paramètre	Valeur
Méthode	GET
URL	{{BASE_URL}}/api/events/

Réponse attendue : 200 OK, tableau JSON

Script Tests :

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("Liste non vide", () => {
  pm.expect(pm.response.json().length).to.be.above(0);
});
```

5.3.3 Test 8 : Détail d'un Événement

Paramètre	Valeur
Méthode	GET
URL	{{BASE_URL}}/api/events/{{EVENT_ID}}/

Réponse attendue : 200 OK, objet JSON de l'événement

5.3.4 Test 9 : Refus de Création (CLIENT)

1. Inscrivez un compte "role": "CLIENT" via Test 1
2. Connectez-vous et récupérez son token
3. Tentez POST /api/events/ avec ce token CLIENT

Réponse attendue : 403 Forbidden

5.4 Tests — Booking Service

5.4.1 Test 10 : Créer une Réservation

Paramètre	Valeur
Méthode	POST
URL	{{BASE_URL}}/api/bookings/
Headers	Content-Type: application/json + Authorization: Bearer {{ACCESS_TOKEN}}

Body :

```
{
  "event_id": "{{EVENT_ID}}",
  "quantity": 2
}
```

Réponse attendue : 201 Created

Script Tests :

```
pm.test("Réservation créée", () => pm.response.to.have.status(201));
pm.test("Status PENDING", () => {
  pm.expect(pm.response.json().status).to.eql("PENDING");
});
```

5.4.2 Test 11 : Mes Réservations

Paramètre	Valeur
Méthode	GET
URL	{{BASE_URL}}/api/bookings/me/
Header	Authorization: Bearer {{ACCESS_TOKEN}}

Réponse attendue : 200 OK, liste JSON des réservations

6 Infrastructure et Déploiement

6.1 Docker Compose

6.1.1 Démarrage complet

```
docker-compose up --build
```

6.1.2 Vérification des conteneurs actifs

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

6.1.3 Services déployés

Service	Image	Ports exposés
traefik	traefik:v2.10	80, 8080
consul	consul:latest	8500
rabbitmq	rabbitmq:3-management	5672, 15672
auth-service	Build local (Django)	via Traefik
event-service	Build local (Django)	via Traefik
booking-service	Build local (Django)	via Traefik
email-worker	Build local (Python)	—
frontend	Build local (React/Nginx)	via Traefik

6.2 Tableau de Bord

Service	URL	Identifiants
Interface Web	http://localhost	—
Traefik Dashboard	http://localhost:8080	—
Consul UI	http://localhost:8500	—
RabbitMQ Console	http://localhost:15672	guest / guest

6.3 Consul — Service Registry

Consul assure la découverte de services dynamique. Chaque microservice s'enregistre au démarrage avec un health check HTTP. Le dashboard permet de visualiser l'état de santé en temps réel.

6.4 Traefik — Reverse Proxy

Préfixe URL	Service cible
/api/auth	auth-service
/api/events	event-service

Préfixe URL	Service cible
/api/bookings	booking-service
/	frontend

La configuration est dynamique via les labels Docker.

6.5 RabbitMQ — Communication Asynchrone

Logs du worker email après une réservation :

```
docker-compose logs -f email-worker
```

Sortie attendue :

```
[x] Envoi d'email : BOOKING_CONFIRMATION à client@billetix.com
```


7 Conclusion

Le projet Billetix implémente avec succès une architecture microservices complète.

Critère	Solution	Résultat
API REST CRUD	Django REST (3 services)	Conforme
Authentification JWT	Auth-Service + bcrypt	Conforme
Interface UI/UX	React + TypeScript	Conforme
Communication asynchrone	RabbitMQ + Worker	Conforme
Service Registry	HashiCorp Consul	Conforme
Reverse Proxy	Traefik v2	Conforme
Déploiement multi-conteneurs	Docker Compose (8 containers)	Conforme

Garanties architecturales :

- **Scalabilité** : chaque service est scalable indépendamment
- **Résilience** : un service défaillant n'impacte pas les autres
- **Découplage** : communication via API REST et RabbitMQ
- **Sécurité** : JWT, bcrypt, protection anti-brute force